

Problem 1)

Results:

```
A is not positive definite
lets make it positive definite
Still not positive definite
Still not positive definite
Still not positive definite
Still not positive definite
Still not positive definite
Still not positive definite
Still not positive definite
Still not positive definite
Still not positive definite
Still not positive definite
Still not positive definite
Still not positive definite
Still not positive definite
2.0480
```

The matrix E is:

```
2.0480      0      0
      0  2.0480      0
      0      0  2.0480
```

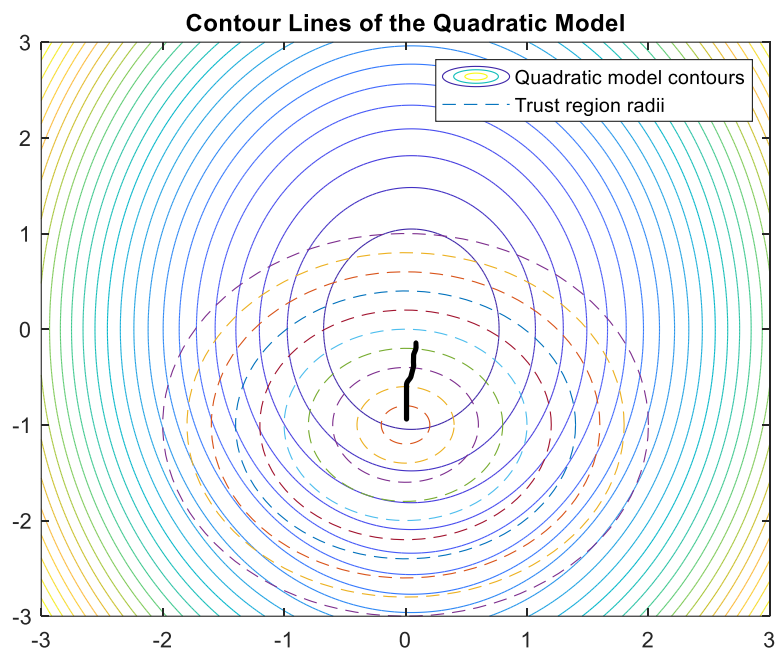
Code:

```
%% problem 1
% algorithm3.4 (cholesky with added multiple of identity)
A = [1 4 3;4 2 5;3 5 3];
beta = 1e-3;
choleskey = false;
% test using chol function
try
    L = chol(A);
catch
    disp('A is not positive definite');
    disp('lets make it positive definite');
end
%make it Positive definite
if min(diag(A))> 0
    taw = 0;
else
    taw = -min(diag(A)) + beta;
end
while~choleskey
    try
        E = taw*eye(3);
        A = A + E;
        L = chol(A);
        choleskey = true;
        disp(taw)
    catch
        disp('Still not positive definite');
    end
    taw_new = max(2*taw,beta);
    taw = taw_new;
end
disp("The matrix E is: ");
disp(E);
```

Note: As we picked different values for beta, you may see that we have different values for tau.

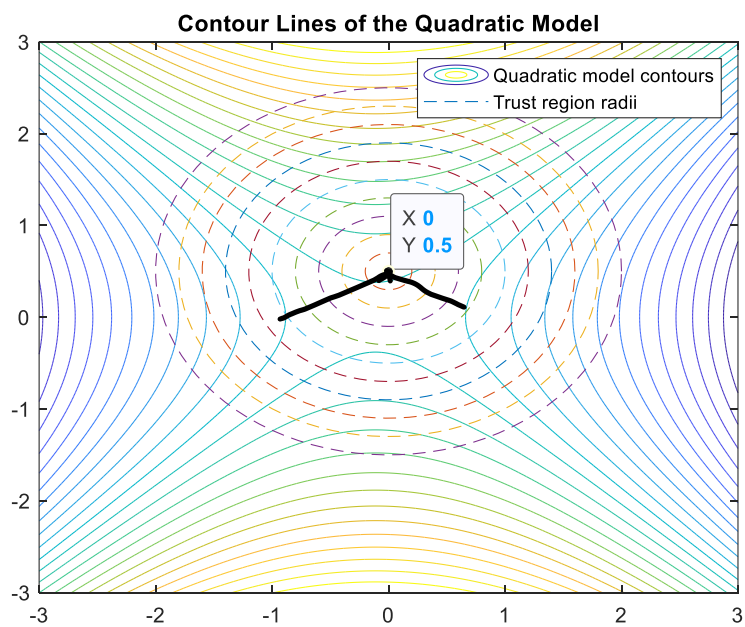
Problem 2)

For point $x = [0, -1]$:



Trust region size determines the step length, and for delta radii around 1, the step length is equal to newton's step.

For point $x = [0, 0.5]$

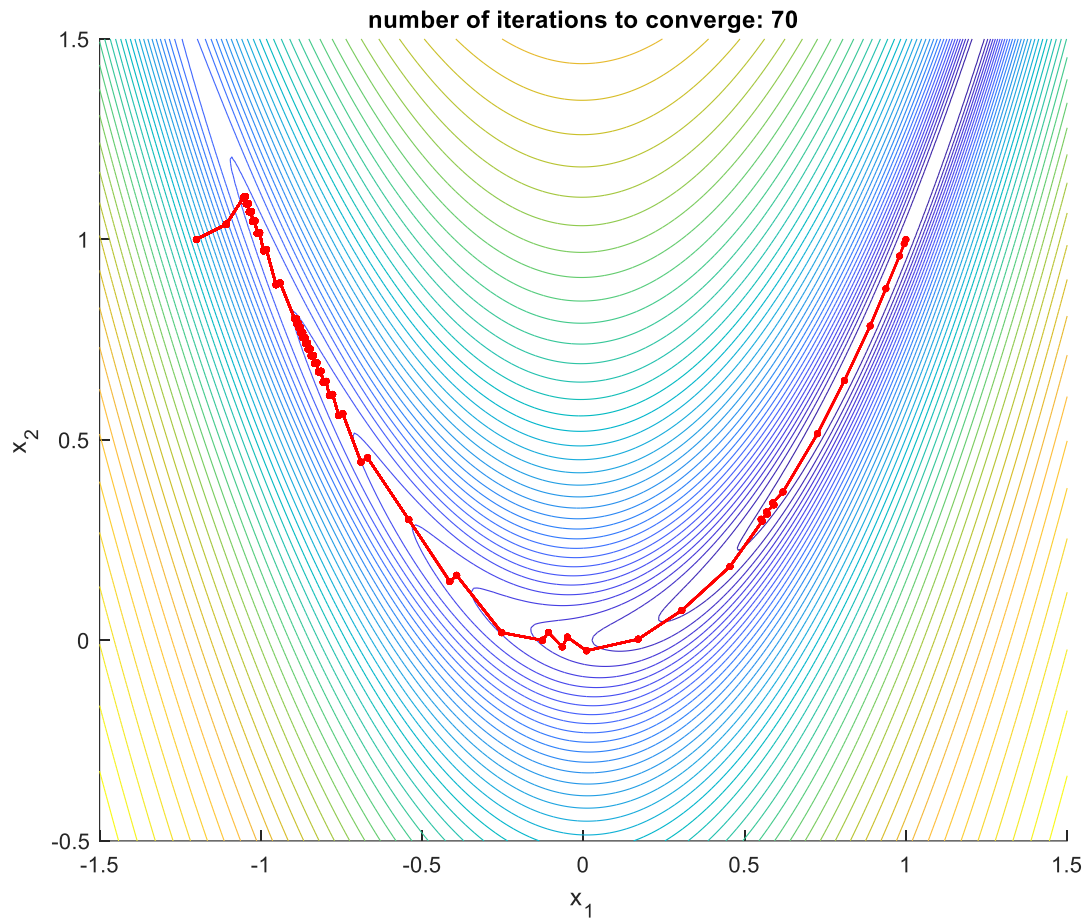


Code:

```
%% problem 2
[x1,x2] = meshgrid(linspace(-3, 3, 400), linspace(-3, 3, 400));
%calculating gradient and hessian at x = [0,-1]
x = [0 -1];
[f,grad,H] = rosenbrock(1,10,x);
mk = zeros(size(x1));
%quadratic model mk(p)
for i = 1:length(x1(:,1))
    for j = 1:length(x1(:,1))
        p = [x1(i,j) - x(1);x2(i,j) - x(2)];
        mk(i,j) = f + grad'*p + 0.5*p'*H*p;
    end
end
% Plotting the contour of the quadratic model
figure;
contour(x1, x2, mk, 25);
title('Contour Lines of the Quadratic Model');
% Plotting trust region different radii
hold on;
for Delta = 0:0.2:2
    theta = linspace(0, 2*pi, 100);
    x_circle = Delta * cos(theta) + x(1);
    y_circle = Delta * sin(theta) + x(2);
    plot(x_circle, y_circle, '--');
end
legend('Quadratic model contours', 'Trust region radii');
hold off;
```

Problem 3)

Converged to:1, 1



Code:

```
%% problem 3
delta_max = 2;
delta = 0.1;
etta = 0.25;
x = [-1.2;1];
tol = 1e-6;
iter = 0;
error = 1;
x_history = x;
[X1, X2] = meshgrid(-1.5:0.01:1.5, -0.5:0.01:1.5);
F = (1 - X1).^2 + 100.*(X2 - X1.^2).^2;
```

```

figure;hold on;
contour(X1, X2, log(F + 2), 45);
xlabel('x_1');
ylabel('x_2');

while error>tol
    %solve for direction
    [f, grad, H] = rosenbrock(1,100,x);
    newton_step = -H\grad;
    S_D_step = - (grad'*H*grad\grad'*grad)*grad;
    taw = delta/norm(S_D_step);
    if norm(newton_step) <= delta
        p = newton_step;
    elseif norm(S_D_step) >= delta
        p = taw*S_D_step;
    else
        taw = find_taw(newton_step,S_D_step,delta);
        p = S_D_step + taw*(newton_step - S_D_step);
    end
    %evaluate the fit
    x_new = x + p;
    x_history = [x_history, x_new];
    plot(x_history(1,:), x_history(2,:), 'r-o', 'LineWidth', 1, 'MarkerFaceColor', 'r','MarkerSize',2);
    title(['number of iterations to converge: ',num2str(iter)]);
    % pause(0.2);

    [f_new,grad_new,~] = rosenbrock(1,100,x_new);
    mk = @(p) f + grad'*p + 0.5*p'*H*p;
    rho = (f - f_new)/(f - mk(p));

    %decide if the delta size is ok or not/adjust it
    if rho< 0.25
        delta_new = 0.25*delta;

    elseif rho >3/4 && norm(p) == delta
        delta_new = min(2*delta,delta_max);
    else
        delta_new = delta;
    end

    %decide to take the step
    if rho > etta
        x = x_new;
        error = norm(grad_new);
    else
        error = norm(grad);

```

```

end
delta = delta_new;
iter = iter + 1;
end
hold off;
disp(['Converged to:',num2str(x_new(1)),', ',num2str(x_new(2))]);
function [f, grad, H] = rosenbrock(a,b,x)
    f = (a - x(1))^2 + b*(x(2) - x(1)^2)^2;
% Gradient of the Rosenbrock function
    if nargin > 1
        grad = [-2*(a - x(1)) - 4*b*x(1)*(x(2) - x(1)^2);
                2*b*(x(2) - x(1)^2)];
    end
% Hessian of the Rosenbrock function
    if nargin > 2
        H = [-4*b*(x(2) - x(1)^2) + 8*b*x(1)^2 + 2, -4*b*x(1);
             -4*b*x(1), 2*b];
    end
end
function taw = find_taw(pU, pB, delta)
    a = norm(pU - pB)^2;
    b = 2*pU'(pU - pB);
    c = norm(pU)^2 - delta^2;
    taw1 = (-b + sqrt(b^2 - 4*a*c))/(2*a);
    taw2 = (-b - sqrt(b^2 - 4*a*c))/(2*a);
    taw = max(taw1,taw2);
    taw = max(0,min(2,taw));
end
% function taw = find_taw(pU, pB, delta)
%     fun = @(taw) norm(pU + taw*(pB - pU)) - delta;
%     taw_initial = 0.5;
%     taw = fzero(fun, taw_initial);
%     taw = max(0, min(2, taw));
% end

```

Note: As I started with a small radius for delta, and smaller max radius for delta, the number of iterations increased. Starting with higher values for delta, helps decreasing number of iterations.

Problem 4:

For the starting point, I created a random vector. Therefore, depending on the starting point, the number of iterations to converge is different. The problem is the n dimensional Rosen Brock function with 20 independent variables. For the bonus problem the number of independent variables increases to 60. In both problems, all the variables converge to 1 (norm of the final vector is 1).

Results:

Iter	, x(1)	,x(2)	f_new	, error	,rho	,delta
	Reached the Trust region boundary					
1	15.97	6.00	4871098.02	478266.62	1.00	0.20
	Reached the Trust region boundary					
2	15.90	6.00	4776170.06	471027.04	1.00	0.20
	Reached the Trust region boundary					
3	15.83	6.01	4682681.92	463868.29	1.00	0.20
	Reached the Trust region boundary					
4	15.77	6.01	4590617.50	456789.73	1.00	0.20
	Reached the Trust region boundary					
5	15.70	6.01	4499960.82	449790.73	1.00	0.40
	Reached the Trust region boundary					
6	15.57	6.01	4322808.32	436026.05	1.01	0.80
	Reached the Trust region boundary					
7	15.30	6.02	3984717.11	409414.35	1.01	0.80
	Reached the Trust region boundary					
8	15.02	6.03	3667429.54	384010.05	1.01	1.60
	Reached the Trust region boundary					
9	14.47	6.05	3091541.05	336636.98	1.03	1.60
	Reached the Trust region boundary					
10	13.91	6.07	2587890.80	293660.62	1.03	1.60
	Reached the Trust region boundary					
11	13.34	6.09	2149674.26	254795.87	1.03	2.00
	Reached the Trust region boundary					
12	12.62	6.12	1684301.04	211569.92	1.04	2.00
	Reached the Trust region boundary					
13	11.89	6.15	1299787.73	173849.04	1.04	2.00
	Reached the Trust region boundary					
14	11.15	6.18	985610.28	141151.69	1.04	2.00

609	1.00	1.00	0.00	0.00	1.95	2.00
610	1.00	1.00	0.00	0.00	1.95	2.00
611	1.00	1.00	0.00	0.00	1.95	2.00
612	1.00	1.00	0.00	0.00	1.95	2.00
613	1.00	1.00	0.00	0.00	1.95	2.00
614	1.00	1.00	0.00	0.00	1.95	2.00
615	1.00	1.00	0.00	0.00	1.95	2.00
616	1.00	1.00	0.00	0.00	1.95	2.00
617	1.00	1.00	0.00	0.00	1.95	2.00
618	1.00	1.00	0.00	0.00	1.95	2.00
619	1.00	1.00	0.00	0.00	1.95	2.00
620	1.00	1.00	0.00	0.00	1.95	2.00

Algorithm Converged

number of iterations to converge: 620

20x1 double		1																	
	1	1.0000																	
	2	1.0000																	
	3	1.0000																	
	4	1.0000																	
	5	1.0000																	
	6	1.0000																	
	7	1.0000																	
	8	1.0000																	
	9	1.0000																	
	10	1.0000																	
	11	1.0000																	
	12	1.0000																	
	13	1.0000																	
	14	1.0000																	
	15	1.0000																	
	16	1.0000																	
	17	1.0000																	
	18	1.0000																	
	19	1.0000																	
	20	1.0000																	

Bonus:

Iter	x(1)	,x(2)	f_new	error	, rho	, delta
------	------	-------	-------	-------	-------	---------

Reached the Trust region boundary

Negative curvature

96	5.57	6.12	101100.02	23189.88	1.07	2.00
----	------	------	-----------	----------	------	------

Reached the Trust region boundary

97	5.09	6.16	61599.89	16542.41	1.09	2.00
----	------	------	----------	----------	------	------

Reached the Trust region boundary

98	4.60	6.21	34166.17	11110.53	1.11	2.00
----	------	------	----------	----------	------	------

Reached the Trust region boundary

99	4.11	6.26	16480.03	6776.92	1.14	2.00
----	------	------	----------	---------	------	------

Reached the Trust region boundary

100	3.59	6.32	6451.08	3491.62	1.21	2.00
-----	------	------	---------	---------	------	------

Reached the trust region boundary

101	3.07	6.39	1966.70	1460.40	1.35	2.00
-----	------	------	---------	---------	------	------

Reached the trust region boundary

102	2.78	6.44	870.13	750.40	1.34	2.00
-----	------	------	--------	--------	------	------

Reached the trust region boundary

103	2.71	6.45	687.45	530.57	1.32	2.00
-----	------	------	--------	--------	------	------

Reached the trust region boundary

104	2.66	6.46	599.11	406.89	1.33	2.00
-----	------	------	--------	--------	------	------

Reached the trust region boundary

105	2.63	6.47	552.61	324.12	1.32	2.00
-----	------	------	--------	--------	------	------

Reached the trust region boundary

106	2.61	6.47	525.23	261.81	1.33	2.00
-----	------	------	--------	--------	------	------

Reached the trust region boundary

107	2.60	6.47	506.25	223.35	1.32	2.00
-----	------	------	--------	--------	------	------

Reached the trust region boundary

Algorithm Converged

number of iterations to converge: 1144

1

	1
35	1.0000
36	1.0000
37	1.0000
38	1.0000
39	1.0000
40	1.0000
41	1.0000
42	1.0000
43	1.0000
44	1.0000
45	1.0000
46	1.0000
47	1.0000
48	1.0000
49	1.0000
50	1.0000
51	1.0000
52	1.0000
53	1.0000
54	1.0000
55	1.0000
56	1.0000
57	1.0000
58	1.0000
59	1.0000
60	1.0000

Note: As I started with a small radius for delta, and smaller max radius for delta, the number of iterations increased. Starting with higher values for delta, helps decreasing number of iterations.